

[apa · free guide]

How I Run 4 Businesses From My Phone Using an AI Brain

The system behind TVE, AthleteOS, Buildout Studios, and AI Pocket Agency.

FORMAT

FREE GUIDE

LENGTH

12 pages

PUBLISHED

2026-05-14

TABLE OF CONTENTS

01	Your AI has amnesia	p.2
02	The moment it changed	p.3
03	What the brain actually is	p.4
04	How the system works day-to-day	p.6
05	Running four lanes at once	p.8
06	The honest catch	p.10
07	What Pocket Agent does	p.11
08	Where to go from here	p.12

SECTION 1.

Your AI has amnesia

Every conversation with your AI starts from zero. No memory of last week's decisions. No recall of what you decided to price at, which contractor you cut, or why you switched the marketing hook. You re-explain your business every single time.

This is not a prompt problem. It's not a model problem. It's an architecture problem.

The way most operators use AI tools — open a chat, type a question, get an answer, close the tab — is structurally the same as hiring a brilliant contractor who forgets the job site the moment they leave. Every Monday morning, you brief them on the whole project again. You re-explain the scope, re-describe the customer, re-share the context. They do great work in that window. Then the window closes.

You feel this when you ask your AI something it should already know. "What did we decide about the pricing tier?" Nothing. "Remind me why we moved away from the old hero copy?" It doesn't have that. "Draft an email to a contractor in the tone I've been using for TVE." It guesses.

The result: you use AI like a glorified search engine. One-shot answers. No continuity. No memory of what your operation actually looks like.

Here's the wall: **most operators don't know what's wrong**. They think the AI just isn't smart enough. They swap models. They try different tools. They start writing longer prompts. None of it fixes it, because the problem isn't intelligence — it's memory.

The brain pattern fixes memory. That's what this guide is about.

SECTION 2.

The moment it changed

I run four businesses. TVE (Tennessee Valley Exteriors) is a roofing and exterior contractor in East Tennessee. AthleteOS is a youth sports platform I built and run. Buildout Studios is the software studio that builds custom systems for other businesses. AI Pocket Agency is this — the AI orchestration layer I built for myself, now packaged as a product.

Running four lanes from one phone used to mean forgetting which lane I was in. A contractor question during a coaching call. A dev question during a roof job. Four sets of context, one brain, no system.

The shift happened when I started treating the AI's context the same way I'd treat a business filing system. Not a chat window. A place where things live so you can find them later.

I created a git repository — `whited-brain` — with one rule: anything worth remembering goes in here as a markdown file. A decision. A voice note transcribed. A competitive finding. The reason I chose one vendor over another. The reason I killed a product direction. The draft voice spec for how I write copy.

The first time my AI cited a decision back to me with a file path and a date, it felt wrong — too good. I asked about pricing strategy on AthleteOS. Instead of guessing, it said: "Per `AOS/AOS_Decision_Log.md` decision #12, dated April 17 — you locked the Parent Pro tier at \$9/mo and ruled out freemium." That's a real answer. Traceable. Correct. No hallucination.

The AI didn't get smarter. It got context. Context is the whole thing.

That sentence sounds simple until you feel the difference. When the AI knows your business — not from this conversation, but because it's been reading a maintained set of files every time it starts — the conversation changes. You stop explaining. You start deciding.

SECTION 3.

What the brain actually is

It's not software. It's a structured directory of markdown files in a git repository. Every agent that touches your business reads those files at the start of every session.

The directory structure is the system. Here's how mine is organized in `whited-brain` :

```
whited-brain/
├── CLAUDE.md          - master context (read first by every agent)
├── AGENTS.md          - behavior rules for agents
├── MEMORY.md          - index of all memory files
├── voice/
│   └── chase-spec.md  - canonical voice spec
├── APA/               - AI Pocket Agency docs
│   ├── Products/
│   ├── Decision_Log.md
│   └── Change_Log.md
├── AOS/               - AthleteOS docs
├── BOS/               - Buildout Studios docs
├── TVE/               - Tennessee Valley Exteriors docs
├── memory/            - indexed memory files
│   ├── project_*.md   - project-level context
│   └── feedback_*.md  - agent behavior feedback
```

Each business gets a directory. Each directory has a decision log, a change log, product specs, and whatever reference material that business needs. The whole thing is one git repo — version-controlled, searchable, durable.

THE FOUR FILES EVERY SESSION READS

CLAUDE.md is the master context file. It tells any AI agent: here's what this repo is, here are the four businesses, here's the current state of each, here are the standing code rules, here's how commits work. It's the briefing document. Every agent reads it before doing anything.

AGENTS.md is the behavior rulebook. No `any` types in TypeScript. No `console.log` in production. No silent catches. Secrets via 1Password only. Additive-only database migrations. These rules are written once and inherited by every agent that ever touches any of my repos.

MEMORY.md is the index. A two-line pointer to every memory file in the system. The memory files themselves are the depth — project context, feedback from prior sessions, standing rules that came from hard lessons. The index is how the AI knows where to look.

`voice/chase-spec.md` is the voice spec. Every agent drafting external copy — emails, drip sequences, landing pages, Skool posts — loads this file first. It tells the agent exactly how I write: cadence rules, banned phrases, sentence structures I favor, worked on-brand versus off-brand examples. The output sounds like me because the spec is thorough.

THE MEMORY FILES

Memory files are persistent facts. When something is worth knowing in the next conversation — not just this one — it gets a file. The file has frontmatter (name, type, description) and a body.

A `project_` file holds current state on a project lane. A `feedback_` file holds a standing rule that came from a real correction. When an agent does something wrong and I correct it, I save the correction as a feedback file so it never happens twice.

The result: feedback from a session on May 12 shapes agent behavior on May 20, even though those are entirely separate conversations.

THE DECISION LOG

Every significant call gets logged. Product decisions. Architecture decisions. Pricing decisions. Each entry has a decision number, a date, and a one-sentence rationale. When I ask the AI later “why did we go with X” — the answer is in the log with the date I made the call.

This matters more than it sounds. The reason behind a decision is the thing that goes stale fastest. The code stays. The PR stays. The reasoning disappears into chat history that nobody reads. The decision log keeps it.

SECTION 4.

How the system works day-to-day

The brain is only useful if you feed it and query it. Here's what both look like in practice, from the job site and from the desk.

Capturing

Capture is the discipline that makes the brain real. Without it, the directory stays empty and the AI is back to guessing.

Three capture patterns I actually use:

Voice to markdown. I'm at a job site. I notice something about the roofing workflow that needs to change — a vendor issue, a new approach to the storm-damage estimate flow. I open a voice memo, say it out loud, transcribe it later (or run it through a transcription tool), and drop it into TVE's directory. Two minutes. Done.

Share-sheet capture. I see a competitor doing something interesting on LinkedIn. I hit Share, the note goes into a staging inbox, and a periodic capture agent formats it and files it in the right place. The thought doesn't die in my clipboard.

End-of-call notes. After a discovery call with a potential BOS client, the follow-up goes in the brain before I write the email. Not the email — the raw context: what they said, what I noticed, what the next move is. The email gets drafted from the brain, not from memory.

Querying

Query is how the brain pays off. I don't open the files manually and read through them. I ask.

The query interface is `bin/brain ask` — a command-line tool that reads the relevant brain files and answers the question with citations:

```
$ bin/brain ask "what did I decide about the AthleteOS pricing tier"
```

The answer comes back with the decision number, the date, and the rationale — pulled directly from the decision log. Not generated. Sourced.

This works for anything stored in the brain:

- › "What's the standing rule on database migrations?" → `AGENTS.md` rule 5: additive-only, never drop without explicit approval.
- › "What's the positioning difference between BOS and APA?" → `BOS/BOS_Studio_Positioning.md` paragraph three, dated May 12.

- › “How do I write in my voice for email?” → `voice/chase-spec.md` § Drip / nurture email pattern.

The key word is **sourced**. The AI isn’t generating a plausible answer. It’s reading files and citing them. That’s a different thing.

Pre-call briefing

Before a call with a BOS prospect, I ask the brain for everything it knows about them. If I’ve had a prior call, the notes are in there. If I researched their business, those notes are in there. If I’ve been tracking a specific pain point they mentioned, it’s in there.

The AI assembles a one-page briefing: who they are, what they said last time, what I need to learn on this call, what the likely close path looks like. I read it for three minutes before I dial.

Contractors who use paper files and a notes app for this spend twenty minutes before every call manually piecing this together. I spend three — because the brain does the retrieval.

REAL EXAMPLE

Before a BOS discovery call in May 2026: I asked the brain for context on a contractor in the storm-restoration space. It pulled their company name from the intake notes, the specific pain they’d described (three spreadsheets, one admin short-staffed during storm season), the HailTrace integration I’d flagged as relevant, and the three questions I should ask to move them toward a close. That was already in the brain from the first call. The second call opened with me saying “last time you mentioned Sandra was solo on the desk during storm season — has that changed?” They said “how do you remember that?” I didn’t explain.

SECTION 5.

Running four lanes at once

The brain isn't just memory. It's the shared context layer that makes it possible to run parallel agents without them losing track of the business.

Running TVE, AthleteOS, Buildout Studios, and AI Pocket Agency simultaneously means four sets of active work at any given point. That work is real: code shipping on AthleteOS, a roofing estimate workflow getting refined at TVE, a BOS client handoff in progress, and APA feature work on the dashboard.

Single-thread AI — one conversation doing one thing — breaks immediately here. The context fills up. You start over. The agent forgets what repo it's in, what the active lane was, what you decided about the pricing model last week.

The brain makes parallel execution work.

WORKTREES

Each code task runs in its own git worktree — an isolated copy of the repo on its own branch. Four agents can work on four different parts of the codebase at the same time without touching each other's files. Lane A rewrites the hero copy. Lane B fixes a broken API route. Lane C drafts the new onboarding flow. None of them know about the others. None of them conflict.

When they're done, they report back. I review. I merge.

STANDING RULES IN THE BRAIN

Every agent that opens any of my repos reads `AGENTS.md` before it does anything. The rules are there: commit conventions, code standards, which things are off-limits, how to verify its own work before reporting done. I wrote those rules once. Every future agent inherits them.

The brain doesn't just answer questions. It shapes behavior. That's the thing most people miss when they first read about this system.

FROM THE PHONE

I'm on a job site with TVE. I have six agents running in parallel on AthleteOS and APA work back at home. I don't need to babysit them.

I check in when they report. I review the SHAs, run a quick verify on what they pushed, and either merge or send feedback. The work moves without me watching it type.

That's what "running businesses from my phone" actually means. Not that the phone is the interface for everything — it's that the system is structured well enough that I don't have to be

at a desk for things to move. The brain holds the context. The agents do the work. I make the calls.

The phone is just the approval surface. The work is already done.

SECTION 6.

The honest catch

The brain pattern is real. It works. I run four businesses out of it. The honest catch is that building it from scratch requires tools most operators don't use and patience most operators don't have.

What the DIY brain requires:

- › A git repository, with basic git literacy (clone, commit, push)
- › Comfort in a terminal — you're running CLI commands daily
- › Markdown, consistently — every note, every decision, every memory file is a `.md` file
- › Discipline to capture — the brain is only as useful as what you put in it
- › Time to structure it — the first two weeks are mostly setup and figuring out what should go where

That's the wall. Not because any of it is insurmountable — it isn't — but because most operators who would benefit from this system are not developers. They're contractors, coaches, agency owners, operators who run real businesses and want AI to help them run better. They do not want to configure a git repo.

I hit this wall too. Not with the git part — I've been coding for years — but with the maintenance. The brain has to be maintained. Dead files, stale context, outdated decisions left uncorrected — they degrade the quality of every answer. Keeping it current is real work.

The \$15 kits at aipocketagency.com teach the pattern as documents. The Dispatch Playbook walks through parallel agent orchestration. The CLAUDE.md Template Library gives you the scaffolding to start your own brain. The Wire-Brain-to-Stack Guide shows how to connect it to your actual tools. Those kits exist for people who want to build this themselves and have the technical foundation to do it.

If you want to build it yourself, start there.

SECTION 7.

What Pocket Agent does

Pocket Agent is the brain pattern wrapped in software. No terminal. No git. No markdown. The same system — minus the 40-hour setup curve and the ongoing maintenance work.

\$97 per month. 14-day free trial. No credit card required to start.

When you sign up, Pocket Agent walks you through an onboarding wizard that configures your brain: your business type, your decision-making patterns, your communication voice, your active projects. It's the same first-time setup I did manually for `whited-brain` — but it takes 20 minutes instead of two weeks.

After setup, the interface is a task launcher. Not a chatbot. A launcher. You tell it what you're working on and it has context. It knows your business, your decisions, your standing rules, your voice. It doesn't need the briefing.

Your memories live in files — the same file-based architecture as `whited-brain`. You own them. When your subscription lapses, the files stay. There's no lock-in to a proprietary format.

The Skool community is included. Every active subscriber gets access to the AI Pocket Agency Skool group — where I do live “build with me” recordings: building the brain pattern on screen, in real business contexts, from scratch. The same trust-building and hands-on learning as a live workshop, inside the subscription.

WHAT IT'S NOT

Pocket Agent is not a general-purpose chatbot. It's not a team product. It's not a project management tool. It's the specific thing — the AI brain pattern for operators — built into software that doesn't require developer skills to use.

If you run one business and want AI to actually remember your business, Pocket Agent is the right tool.

If you run multiple businesses and want the context isolation that the brain pattern gives you across lanes, Pocket Agent is the right tool.

If you want to use Claude or GPT-4o or whatever the next model is and have it know your business every time, without briefing it from scratch every session — Pocket Agent is the right tool.

SECTION 8.

Where to go from here

Two paths. Pick based on where you are right now.

If you want software: Start your 14-day free trial at app.aipocketagency.com. No credit card. The onboarding wizard takes 20 minutes. You'll have a working brain by the end of the first session.

If you want to build it yourself: Start with the CLAUDE.md Template Library — the \$15 kit that gives you the base scaffolding and walks through how to fill it for your specific business. Then grab the Dispatch Playbook if you want to run parallel agents. The full kit catalog is at aipocketagency.com.

Either way: the thing that changes when you use this system is not speed. It's continuity. The AI stops being a fresh-start tool and starts being a system that knows your business. That's the shift. Everything downstream of that shift is better work.

— Chase

LINKS

Pocket Agent trial	app.aipocketagency.com
\$15 kits	aipocketagency.com
Skool community	skool.com/aipocketagency